EXPERIMENT 3: DECISION TREE (ID3) ALGORITHM

EXPERIMENT QUESTION: DEMONSTRATE THE WORKING OF THE DECISION TREE BASED ID3 ALGORITHM. USE AN APPROPRIATE DATA SET FOR BUILDING THE DECISION TREE AND APPLY THIS KNOWLEDGE TO CLASSIFY A NEW SAMPLE.

1. Aim

To build an ID3 decision tree that predicts whether tennis will be played from weather conditions, and to classify a new day.

2. Dataset Description

The training examples are shown below. The last column is the target class.

Outlook	Temperature	Humidity	Wind	Play Tennis
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

3. Algorithm

1. Calculate the entropy of the target attribute for the current set of examples.

2. For every remaining attribute, calculate the information gain.
3. Select the attribute with the highest information gain as the decision node.
4. Create one branch for each value of the selected attribute and divide the examples accordingly.
5. If all examples in a branch belong to one class, create a leaf node with that class.
6. Otherwise, repeat steps 1–4 on the branch with the remaining attributes (use the majority class if no attributes remain).
7. Classify the new sample by traversing the tree from the root to a leaf.

PROGRAM

```
import math
from collections import Counter

attributes = ['Outlook', 'Temperature', 'Humidity', 'Wind']
data = [
    ['Sunny', 'Hot', 'High', 'Weak', 'No'],
    ['Sunny', 'Hot', 'High', 'Strong', 'No'],
    ['Overcast', 'Hot', 'High', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'High', 'Weak', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'Strong', 'No'],
    ['Overcast', 'Cool', 'Normal', 'Strong', 'Yes'],
    ['Sunny', 'Mild', 'High', 'Weak', 'No'],
    ['Sunny', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'Normal', 'Weak', 'Yes'],
    ['Sunny', 'Mild', 'Normal', 'Strong', 'Yes'],
    ['Overcast', 'Mild', 'High', 'Strong', 'Yes'],
    ['Overcast', 'Hot', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'High', 'Strong', 'No']
]

def entropy(rows):
    total = len(rows)
    counts = Counter(r[-1] for r in rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

def info_gain(rows, idx):
    total = len(rows)
    remainder = 0
    for v in set(r[idx] for r in rows):
        subset = [r for r in rows if r[idx] == v]
        remainder += len(subset) / total * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attr_idx):
    classes = [r[-1] for r in rows]
    if len(set(classes)) == 1:
        return classes[0]
    if not attr_idx:
        return Counter(classes).most_common(1)[0][0]
    best = max(attr_idx, key=lambda a: info_gain(rows, a))
    tree = {attributes[best]: {}}
    for v in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == v]
        rest = [a for a in attr_idx if a != best]
        tree[attributes[best]][v] = id3(subset, rest)
```

```
    return tree

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "->", tree)
        return
    attr = next(iter(tree))
    for value, subtree in tree[attr].items():
        print(f"{indent}[{attr} = {value}]")
        print_tree(subtree, indent + "    ")

def classify(tree, sample):
    while isinstance(tree, dict):
        attr = next(iter(tree))
        tree = tree[attr].get(sample[attr])
        if tree is None:
            return "Unknown"
    return tree

all_idx = list(range(len(attributes)))
print("Entropy of the full data set: %.4f" % entropy(data))
for a in all_idx:
    print("Gain(%s) = %.4f" % (attributes[a], info_gain(data, a)))

tree = id3(data, all_idx)
print("\nDecision Tree:")
print_tree(tree)

new_sample = {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind':
'Strong'}
print("\nNew sample:", new_sample)
print("Predicted class:", classify(tree, new_sample))
```

OUTPUT

```
========== RESTART: C:/Users/Solo Pavan/pavan1.py ==========
Entropy of the full data set: 0.9403
Gain(Outlook) = 0.2467
Gain(Temperature) = 0.0292
Gain(Humidity) = 0.1518
Gain(Wind) = 0.0481

Decision Tree:
[Outlook = Overcast]
    -> Yes
[Outlook = Rain]
    [Wind = Strong]
        -> No
    [Wind = Weak]
        -> Yes
[Outlook = Sunny]
    [Humidity = High]
        -> No
    [Humidity = Normal]
        -> Yes

New sample: {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Win
d': 'Strong'}
Predicted class: No
```

Subproblem 3.2: Flu Diagnosis

1. Aim

To use ID3 to diagnose whether a patient has Flu from fever, cough, and body ache.

2. **Dataset Description**

The training examples are shown below. The last column is the target class.

Fever	Cough	Body Ache	Diagnosis
High	Yes	Yes	Flu
High	Yes	No	Flu
High	No	Yes	Flu
High	No	No	Flu
High	No	No	No Flu
Normal	Yes	Yes	Flu
Normal	Yes	No	No Flu
Normal	No	Yes	No Flu
Normal	No	No	No Flu
Normal	Yes	No	No Flu
High	Yes	Yes	Flu
Normal	No	No	No Flu

3. Algorithm

1. Calculate the entropy of the target attribute for the current set of examples.
2. For every remaining attribute, calculate the information gain.
3. Select the attribute with the highest information gain as the decision node.
4. Create one branch for each value of the selected attribute and divide the examples accordingly.
5. If all examples in a branch belong to one class, create a leaf node with that class.
6. Otherwise, repeat steps 1–4 on the branch with the remaining attributes (use the majority class if no attributes remain).
7. Classify the new sample by traversing the tree from the root to a leaf.

PROGRAM

```
import math
from collections import Counter

attributes = ['Fever', 'Cough', 'Body Ache']
data = [
    ['High', 'Yes', 'Yes', 'Flu'],
    ['High', 'Yes', 'No', 'Flu'],
    ['High', 'No', 'Yes', 'Flu'],
```

```
    ['High', 'No', 'No', 'Flu'],
    ['High', 'No', 'No', 'No Flu'],
    ['Normal', 'Yes', 'Yes', 'Flu'],
    ['Normal', 'Yes', 'No', 'No Flu'],
    ['Normal', 'No', 'Yes', 'No Flu'],
    ['Normal', 'No', 'No', 'No Flu'],
    ['Normal', 'Yes', 'No', 'No Flu'],
    ['High', 'Yes', 'Yes', 'Flu'],
    ['Normal', 'No', 'No', 'No Flu']
]

def entropy(rows):
    total = len(rows)
    counts = Counter(r[-1] for r in rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

def info_gain(rows, idx):
    total = len(rows)
    remainder = 0
    for v in set(r[idx] for r in rows):
        subset = [r for r in rows if r[idx] == v]
        remainder += len(subset) / total * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attr_idx):
    classes = [r[-1] for r in rows]
    if len(set(classes)) == 1:
        return classes[0]
    if not attr_idx:
        return Counter(classes).most_common(1)[0][0]
    best = max(attr_idx, key=lambda a: info_gain(rows, a))
    tree = {attributes[best]: {}}
    for v in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == v]
        rest = [a for a in attr_idx if a != best]
        tree[attributes[best]][v] = id3(subset, rest)
    return tree

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "->", tree)
        return
    attr = next(iter(tree))
    for value, subtree in tree[attr].items():
        print(f"{indent}[{attr} = {value}]")
        print_tree(subtree, indent + "    ")

def classify(tree, sample):
    while isinstance(tree, dict):
        attr = next(iter(tree))
        tree = tree[attr].get(sample[attr])
        if tree is None:
            return "Unknown"
    return tree

all_idx = list(range(len(attributes)))
print("Entropy of the full data set: %.4f" % entropy(data))
for a in all_idx:
    print("Gain(%s) = %.4f" % (attributes[a], info_gain(data, a)))

tree = id3(data, all_idx)
print("\nDecision Tree:")
print_tree(tree)

new_sample = {'Fever': 'Normal', 'Cough': 'Yes', 'Body Ache': 'Yes'}
print("\nNew sample:", new_sample)
print("Predicted class:", classify(tree, new_sample))
```

OUTPUT

```
========== RESTART: C:/Users/Solo Pavan/PAVANS.py ==========
Entropy of the full data set: 1.0000
Gain(Fever) = 0.3500
Gain(Cough) = 0.0817
Gain(Body Ache) = 0.1957

Decision Tree:
[Fever = High]
    [Cough = No]
        [Body Ache = No]
            -> Flu
        [Body Ache = Yes]
            -> Flu
    [Cough = Yes]
        -> Flu
[Fever = Normal]
    [Body Ache = No]
        -> No Flu
    [Body Ache = Yes]
        [Cough = No]
            -> No Flu
        [Cough = Yes]
            -> Flu

New sample: {'Fever': 'Normal', 'Cough': 'Yes', 'Body Ache': 'Yes'}
Predicted class: Flu
```

Subproblem 3.3: Loan Approval Prediction

1. Aim

To build a decision tree that predicts whether a bank loan application is approved.

2. Dataset Description

The training examples are shown below. The last column is the target class.

Income	Employment	Credit History	Loan Approved
High	Employed	Good	Yes
High	Unemployed	Good	Yes
Medium	Employed	Good	Yes
Medium	Unemployed	Good	Yes
Low	Employed	Good	Yes
Low	Unemployed	Good	No
High	Employed	Poor	Yes
High	Unemployed	Poor	No
Medium	Employed	Poor	No
Medium	Unemployed	Poor	No
Low	Employed	Poor	No
Low	Unemployed	Poor	No

3. Algorithm

1. Calculate the entropy of the target attribute for the current set of examples.
2. For every remaining attribute, calculate the information gain.
3. Select the attribute with the highest information gain as the decision node.
4. Create one branch for each value of the selected attribute and divide the examples accordingly.
5. If all examples in a branch belong to one class, create a leaf node with that class.
6. Otherwise, repeat steps 1–4 on the branch with the remaining attributes (use the majority class if no attributes remain).
7. Classify the new sample by traversing the tree from the root to a leaf.

PROGRAM

```
import math
from collections import Counter

attributes = ['Income', 'Employment', 'Credit History']
data = [
    ['High', 'Employed', 'Good', 'Yes'],
    ['High', 'Unemployed', 'Good', 'Yes'],
    ['Medium', 'Employed', 'Good', 'Yes'],
```

```
    ['Medium', 'Unemployed', 'Good', 'Yes'],
    ['Low', 'Employed', 'Good', 'Yes'],
    ['Low', 'Unemployed', 'Good', 'No'],
    ['High', 'Employed', 'Poor', 'Yes'],
    ['High', 'Unemployed', 'Poor', 'No'],
    ['Medium', 'Employed', 'Poor', 'No'],
    ['Medium', 'Unemployed', 'Poor', 'No'],
    ['Low', 'Employed', 'Poor', 'No'],
    ['Low', 'Unemployed', 'Poor', 'No']
]

def entropy(rows):
    total = len(rows)
    counts = Counter(r[-1] for r in rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

def info_gain(rows, idx):
    total = len(rows)
    remainder = 0
    for v in set(r[idx] for r in rows):
        subset = [r for r in rows if r[idx] == v]
        remainder += len(subset) / total * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attr_idx):
    classes = [r[-1] for r in rows]
    if len(set(classes)) == 1:
        return classes[0]
    if not attr_idx:
        return Counter(classes).most_common(1)[0][0]
    best = max(attr_idx, key=lambda a: info_gain(rows, a))
    tree = {attributes[best]: {}}
    for v in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == v]
        rest = [a for a in attr_idx if a != best]
        tree[attributes[best]][v] = id3(subset, rest)
    return tree

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "->", tree)
        return
    attr = next(iter(tree))
    for value, subtree in tree[attr].items():
        print(f"{indent}[{attr} = {value}]")
        print_tree(subtree, indent + "    ")

def classify(tree, sample):
    while isinstance(tree, dict):
        attr = next(iter(tree))
        tree = tree[attr].get(sample[attr])
        if tree is None:
            return "Unknown"
    return tree

all_idx = list(range(len(attributes)))
print("Entropy of the full data set: %.4f" % entropy(data))
for a in all_idx:
    print("Gain(%s) = %.4f" % (attributes[a], info_gain(data, a)))

tree = id3(data, all_idx)
print("\nDecision Tree:")
print_tree(tree)

new_sample = {'Income': 'Medium', 'Employment': 'Employed', 'Credit History': 'Poor'}
print("\nNew sample:", new_sample)
print("Predicted class:", classify(tree, new_sample))
```

OUTPUT

```
====================== RESTART: C:/Users/Solo Pavan/PAVAN.py ====================
Entropy of the full data set: 1.0000
Gain(Income) = 0.1258
Gain(Employment) = 0.0817
Gain(Credit History) = 0.3500

Decision Tree:
[Credit History = Good]
    [Income = High]
        -> Yes
    [Income = Low]
        [Employment = Employed]
            -> Yes
        [Employment = Unemployed]
            -> No
    [Income = Medium]
        -> Yes
[Credit History = Poor]
    [Income = High]
        [Employment = Employed]
            -> Yes
        [Employment = Unemployed]
            -> No
    [Income = Low]
        -> No
    [Income = Medium]
        -> No

New sample: {'Income': 'Medium', 'Employment': 'Employed', 'Credit History': 'Po
or'}
Predicted class: No
```

Subproblem 3.4: Email Spam Detection

1. Aim

To use ID3 to classify an email as Spam or Not Spam from simple message features.

2. Dataset Description

The training examples are shown below. The last column is the target class.

Contains Link	Unknown Sender	All Caps Subject	Class
Yes	Yes	Yes	Spam
Yes	Yes	No	Spam
Yes	No	Yes	Spam
Yes	No	No	Spam
Yes	No	No	Not Spam
Yes	Yes	Yes	Spam
No	Yes	Yes	Spam
No	Yes	No	Not Spam
No	No	Yes	Not Spam
No	No	No	Not Spam
No	No	No	Not Spam
No	Yes	No	Not Spam

3. Algorithm

1. Calculate the entropy of the target attribute for the current set of examples.
2. For every remaining attribute, calculate the information gain.
3. Select the attribute with the highest information gain as the decision node.
4. Create one branch for each value of the selected attribute and divide the examples accordingly.
5. If all examples in a branch belong to one class, create a leaf node with that class.
6. Otherwise, repeat steps 1–4 on the branch with the remaining attributes (use the majority class if no attributes remain).
7. Classify the new sample by traversing the tree from the root to a leaf.

PROGRAM

```
import math
from collections import Counter

attributes = ['Contains Link', 'Unknown Sender', 'All Caps Subject']
data = [
    ['Yes', 'Yes', 'Yes', 'Spam'],
    ['Yes', 'Yes', 'No', 'Spam'],
    ['Yes', 'No', 'Yes', 'Spam'],
```

```
    ['Yes', 'No', 'No', 'Spam'],
    ['Yes', 'No', 'No', 'Not Spam'],
    ['Yes', 'Yes', 'Yes', 'Spam'],
    ['No', 'Yes', 'Yes', 'Spam'],
    ['No', 'Yes', 'No', 'Not Spam'],
    ['No', 'No', 'Yes', 'Not Spam'],
    ['No', 'No', 'No', 'Not Spam'],
    ['No', 'No', 'No', 'Not Spam'],
    ['No', 'Yes', 'No', 'Not Spam']
]

def entropy(rows):
    total = len(rows)
    counts = Counter(r[-1] for r in rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

def info_gain(rows, idx):
    total = len(rows)
    remainder = 0
    for v in set(r[idx] for r in rows):
        subset = [r for r in rows if r[idx] == v]
        remainder += len(subset) / total * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attr_idx):
    classes = [r[-1] for r in rows]
    if len(set(classes)) == 1:
        return classes[0]
    if not attr_idx:
        return Counter(classes).most_common(1)[0][0]
    best = max(attr_idx, key=lambda a: info_gain(rows, a))
    tree = {attributes[best]: {}}
    for v in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == v]
        rest = [a for a in attr_idx if a != best]
        tree[attributes[best]][v] = id3(subset, rest)
    return tree

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "->", tree)
        return
    attr = next(iter(tree))
    for value, subtree in tree[attr].items():
        print(f"{indent}[{attr} = {value}]")
        print_tree(subtree, indent + "    ")

def classify(tree, sample):
    while isinstance(tree, dict):
        attr = next(iter(tree))
        tree = tree[attr].get(sample[attr])
        if tree is None:
            return "Unknown"
    return tree

all_idx = list(range(len(attributes)))
print("Entropy of the full data set: %.4f" % entropy(data))
for a in all_idx:
    print("Gain(%s) = %.4f" % (attributes[a], info_gain(data, a)))

tree = id3(data, all_idx)
print("\nDecision Tree:")
print_tree(tree)

new_sample = {'Contains Link': 'Yes', 'Unknown Sender': 'No', 'All Caps Subject':
'Yes'}
print("\nNew sample:", new_sample)
print("Predicted class:", classify(tree, new_sample))
```

OUTPUT

```
====================== RESTART: C:/Users/Solo Pavan/PAVAN.py ====================
Entropy of the full data set: 1.0000
Gain(Contains Link) = 0.3500
Gain(Unknown Sender) = 0.0817
Gain(All Caps Subject) = 0.1957

Decision Tree:
[Contains Link = No]
    [All Caps Subject = No]
        -> Not Spam
    [All Caps Subject = Yes]
        [Unknown Sender = No]
            -> Not Spam
        [Unknown Sender = Yes]
            -> Spam
[Contains Link = Yes]
    [Unknown Sender = No]
        [All Caps Subject = No]
            -> Spam
        [All Caps Subject = Yes]
            -> Spam
    [Unknown Sender = Yes]
        -> Spam

New sample: {'Contains Link': 'Yes', 'Unknown Sender': 'No', 'All Caps Subject':
 'Yes'}
Predicted class: Spam
```

Subproblem 3.5: Student Placement Prediction

1. Aim

To build an ID3 tree that predicts whether a student is Placed from academic and skill attributes.

2. Dataset Description

The training examples are shown below. The last column is the target class.

CGPA	Internship	Communication	Placement
High	Yes	Good	Placed
High	Yes	Average	Placed
High	No	Good	Placed
High	No	Average	Not Placed
Medium	Yes	Good	Placed
Medium	Yes	Average	Not Placed
Medium	No	Good	Not Placed
Medium	No	Average	Not Placed
Low	Yes	Good	Not Placed
Low	No	Average	Not Placed
Low	Yes	Average	Not Placed
High	Yes	Average	Placed

3. Algorithm

1. Calculate the entropy of the target attribute for the current set of examples.
2. For every remaining attribute, calculate the information gain.
3. Select the attribute with the highest information gain as the decision node.
4. Create one branch for each value of the selected attribute and divide the examples accordingly.
5. If all examples in a branch belong to one class, create a leaf node with that class.
6. Otherwise, repeat steps 1–4 on the branch with the remaining attributes (use the majority class if no attributes remain).
7. Classify the new sample by traversing the tree from the root to a leaf.

PROGRAM

```
import math
from collections import Counter

attributes = ['CGPA', 'Internship', 'Communication']
data = [
    ['High', 'Yes', 'Good', 'Placed'],
```

```
    ['High', 'Yes', 'Average', 'Placed'],
    ['High', 'No', 'Good', 'Placed'],
    ['High', 'No', 'Average', 'Not Placed'],
    ['Medium', 'Yes', 'Good', 'Placed'],
    ['Medium', 'Yes', 'Average', 'Not Placed'],
    ['Medium', 'No', 'Good', 'Not Placed'],
    ['Medium', 'No', 'Average', 'Not Placed'],
    ['Low', 'Yes', 'Good', 'Not Placed'],
    ['Low', 'No', 'Average', 'Not Placed'],
    ['Low', 'Yes', 'Average', 'Not Placed'],
    ['High', 'Yes', 'Average', 'Placed']
]

def entropy(rows):
    total = len(rows)
    counts = Counter(r[-1] for r in rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

def info_gain(rows, idx):
    total = len(rows)
    remainder = 0
    for v in set(r[idx] for r in rows):
        subset = [r for r in rows if r[idx] == v]
        remainder += len(subset) / total * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attr_idx):
    classes = [r[-1] for r in rows]
    if len(set(classes)) == 1:
        return classes[0]
    if not attr_idx:
        return Counter(classes).most_common(1)[0][0]
    best = max(attr_idx, key=lambda a: info_gain(rows, a))
    tree = {attributes[best]: {}}
    for v in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == v]
        rest = [a for a in attr_idx if a != best]
        tree[attributes[best]][v] = id3(subset, rest)
    return tree

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "->", tree)
        return
    attr = next(iter(tree))
    for value, subtree in tree[attr].items():
        print(f"{indent}[{attr} = {value}]")
        print_tree(subtree, indent + "    ")

def classify(tree, sample):
    while isinstance(tree, dict):
        attr = next(iter(tree))
        tree = tree[attr].get(sample[attr])
        if tree is None:
            return "Unknown"
    return tree

all_idx = list(range(len(attributes)))
print("Entropy of the full data set: %.4f" % entropy(data))
for a in all_idx:
    print("Gain(%s) = %.4f" % (attributes[a], info_gain(data, a)))

tree = id3(data, all_idx)
print("\nDecision Tree:")
print_tree(tree)

new_sample = {'CGPA': 'Medium', 'Internship': 'Yes', 'Communication': 'Good'}
print("\nNew sample:", new_sample)
print("Predicted class:", classify(tree, new_sample))
```

OUTPUT

```
====================== RESTART: C:/Users/Solo Pavan/PAVAN.py ==================
Entropy of the full data set: 0.9799
Gain(CGPA) = 0.4086
Gain(Internship) = 0.1043
Gain(Communication) = 0.0718

Decision Tree:
[CGPA = High]
    [Internship = No]
        [Communication = Average]
            -> Not Placed
        [Communication = Good]
            -> Placed
    [Internship = Yes]
        -> Placed
[CGPA = Low]
    -> Not Placed
[CGPA = Medium]
    [Internship = No]
        -> Not Placed
    [Internship = Yes]
        [Communication = Average]
            -> Not Placed
        [Communication = Good]
            -> Placed

New sample: {'CGPA': 'Medium', 'Internship': 'Yes', 'Communication': 'Good'}
Predicted class: Placed
```